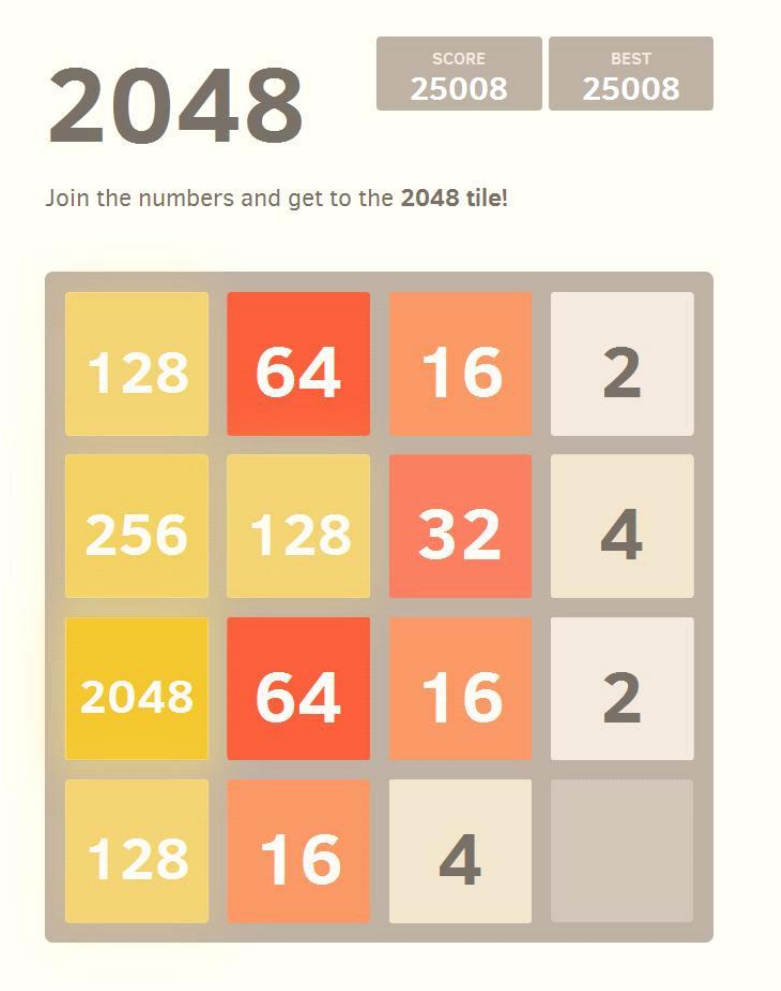
P01 – 2048 game playing agent

1. Background



The game 2048 was developed by Italian programmer Gabriele Cirulli during a week-end, based on 1024 by Veewo Studio and conceptually similar to Threes by Asher Vollmer (you may read his story of the making & rip-offs in game development here: http://asherv.com/threes/threemails/).

The gameplay for 2048 is as follows: You can move the tiles (all of them at once) with your arrow keys. When two tiles with the same number touch, they merge into one, with the new tile having twice the old value. After each board-changing move, a new random

tile spawns, having a value of 2 (90% chance) or 4 (10% chance). You win by creating a tile with the value 2048.

In this lab, you will develop a simple (part 3) as well as a sophisticated AI agent (part 4) to remote control the 2048 game running in your browser. Before, make sure you have all the prerequisites set up (part 2).

2. Prerequisites

2.1 Python

Throughout this course, we will be using Python and trust you have done so before. If not, you can find a ready-to-go distribution that offers everything we need in Anaconda (you need to register free of charge, though):

- Download the correct version for your system from here (several hundred MB): https://www.anaconda.com/download
- Be sure to use the Version for Python 3.x, *not* Python 2.x
- Take the 64-bit variant if you have a 64-bit system

Anaconda (Python) needs to be up to date to be used for the labs accompanying this module. Please update it using either the Anaconda Launcher or the following two console commands (from Anaconda install directory):

```
conda update conda
conda update anaconda
```

If you want to brush up your Python, the following links help get you started (in order of increasing sophistication):

- Beginner's cheat sheet: https://groklearning-cdn.com/resources/cheatsheet-python-1.pdf
- Programmer's cheat sheet: https://scouv.lisn.upsaclay.fr/python-memento/memento-python3-en-latest.pdf
- Official Python 3 tutorial: https://docs.python.org/3/tutorial/
- Python for data science: https://www.analyticsvidhya.com/courses/introduction-to-data-science/

2.1 Configuring your browser

To play the game of 2048, you should use Chrome (other solutions are possible but neither recommended nor supported).

1. Download and install Chrome from https://www.google.com/chrome/ (if it is not already installed)

2. To be able to remote control the browser, you need to start Chrome with the command line argument `--remote-debugging-port=9222`
   On Windows:
   - create a new shortcut on your desktop to the Chrome browser
   - Right click the shortcut, open "Properties"
   - In the field "Target", add `--remote-debugging-port=9222` --remote-allow-origins=* or `--remote-debugging-port=9222 --remote-allow-origins=http://localhost:9222 --user-data-dir="$HOME/chrome-dev-session"` at the end

   On Mac:
   - Turn off all tabs and related tasks in Google Chrome and quit Google Chrome
   - Run the following command in the terminal to set the remote debugging port and allow visit from localhost: `/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome --remote-debugging-port=9222 --remote-allow-origins=http://localhost:9222`

   *(Make sure to **check the text if you copy&paste** it from here: sometimes, hyphens etc. are messed up!)*

3. Install the required dependency "websocket-client" in your Python environment:
   `pip install websocket-client`

4. Start Chrome with the command line argument (see above), and open the website of the game: https://classic.play2048.co/

You are now able to run the Python code templates provided with this lab description, which will control the browser game.

3. Getting started with 2048

Be sure that your Chrome browser is open (with use of command line parameters activated) and correctly configured (see previous section), otherwise it will fail.

3.1 Running the code template `2048.py`

To run the Python script, you have to open your console/terminal and write
`python 2048.py -b chrome`

3.2 Modifying the code template `heuristicai.py`

To program your first own agent that will solve the game you have to modify the file `heuristicai.py`.

The goal of this task is to build an agent which is (at the minimum) better than a random player, by coming up with some rules/heuristics *apart from the algorithms treated in the lecture*. This task could also be abbreviated as *"create an AI agent manually"* – don't use AI you learned about, but hard-code your own smartness.

To do so you, have to implement the method `find_best_move(board)` in a better way than it is at the moment, where it will just move the board in a random way. You are provided with a 2D list of the game state, and you have to return the direction in which the game should move.

- The parameter `board` of `find_best_move` is a numpy 2D matrix which you can access like a normal 2D array. It represents the current gamestate.
- The sample solution achieves between 7'000 and 12'000 points and in most of the cases a 512/1024 tile as its maximum.

Hints: Some ideas for useful heuristics are

- When many tiles are on the board, the chance that you can't move anymore is greatly increased; thus, one of your goals is to have the board as empty as possible.
- If tiles of big value are at the corner of the board, they don't block the merging of the "smaller" tiles.
- A move bringing two tiles with the same value next to each other is preferable over a move that won't give you this advantage.

Can you come up with some rules to master the game? C'mon!

Before you continue with the next task, make a self-assessment: How well does your agent play? How does it compare to your classmates? To results you see on the web (if your search for "2048 AI")? How does it compare to you as a human player (not only result-wise, but also if you compare the choice of moves)? What are reasons that explain what you observe? What makes implementing human game-playing (depending on your achieved results) particularly easy or hard?

4. AI for 2048

In this task your goal is to build a game-playing 2048 agent based on the expectimax algorithm. To do so, you will modify the script `searchai.py`. Additionally, please modify 2048.py so that it calls `find_best_move(board)` from `searchai.py`, not `heuristicai.py` as in the previous task.

To implement expectimax, you still need a good heuristic to score a current board. You can re-use ideas (or code) here from what you tried in the previous task or implement others. Probably a big difference to your previous implementation will be that now your heuristics are combined with proven and effective systematic search capability.

[Optional: If you are not satisfied with your own heuristics, check this post for some ideas: http://stackoverflow.com/questions/22342854/what-is-the-optimal-algorithm-for-the-game-2048. It also provides you with other algorithms and ideas to beat the game. If you are interested, give it a try!]

Assess the performance of your solution: What makes the difference to your agent of the previous task (if any)? What differentiates it from a human player? From state of the art (see e.g. http://www.cs.put.poznan.pl/mszubert/pub/szubert2014cig.pdf)? You can compete with your peers using the following online leaderboard: https://goo.gl/meh3Ro

Hints: How to use expectimax for 2048

The main challenge in playing 2048 is the randomness of where and which kind of tile will spawn. This makes the game nondeterministic. Expectimax is a good choice for games which have a nondeterministic model of operation.

Figure 1 shows an excerpt of how expectimax will work with `DEPTH=2`. Recall that after a successful move (which alters the board), a new tile will spawn on an empty field, with the chance of a value-2 tile is 90% that of a value-4 tile is 10%. Therefore, when you have three empty fields after the move, you can have six possible outcomes as new board states after the subsequent spawning (as depicted in Figure 1).

The heuristic to score the board is only needed at the leafs of the tree. The sample solution can reach 2048 most of the time with DEPTH=2.

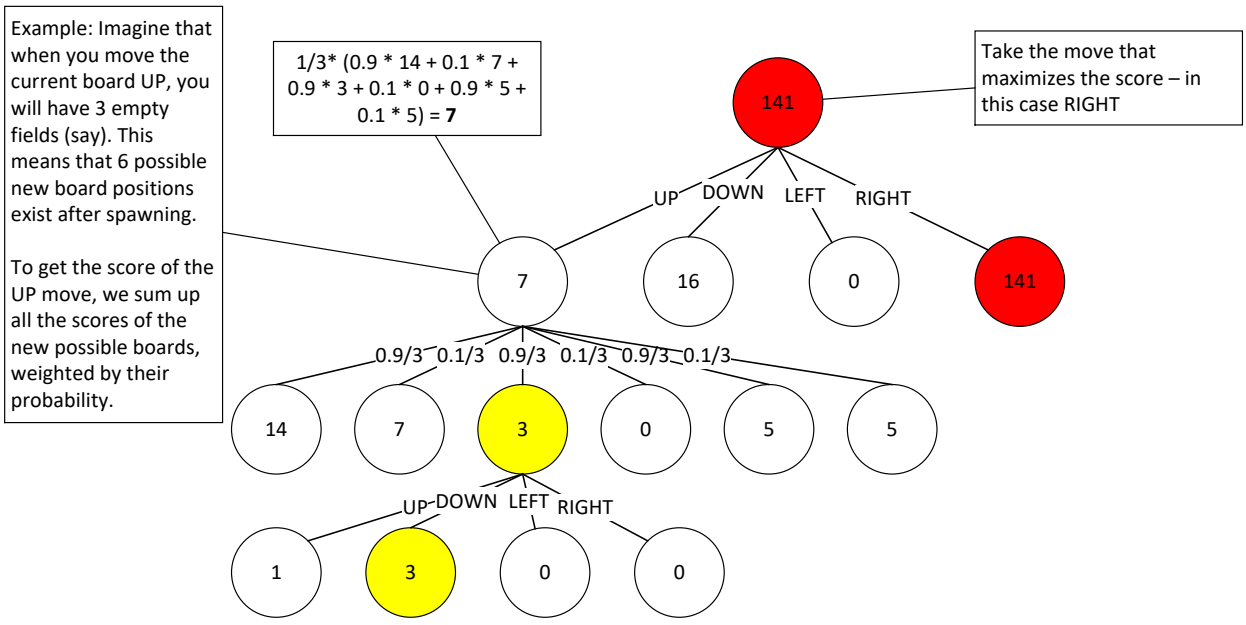


Figure 1: A probabilistic search tree for 2048.